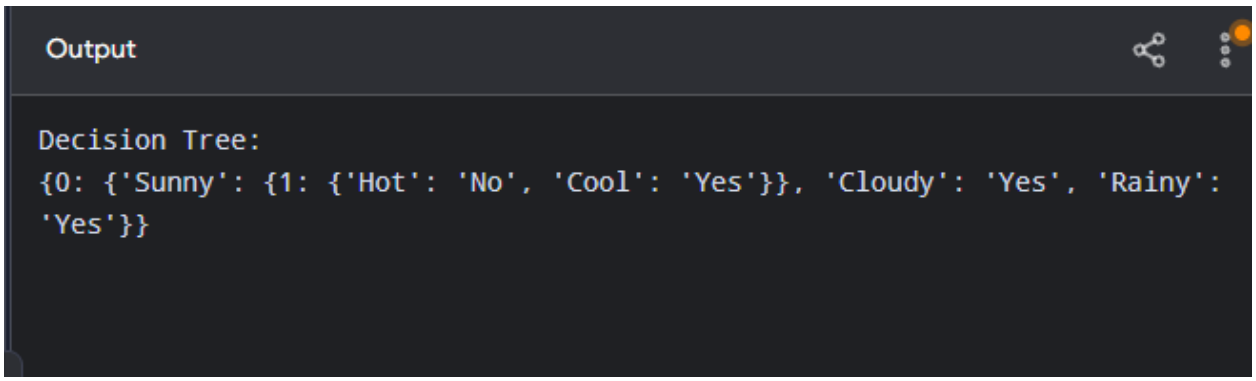
Lab experiment

Decision tree

Coding:

```
# Decision Tree using ID3 Algorithm

import math
from collections import Counter

# Dataset: Outlook, Temperature, Play
data = [
    ['Sunny', 'Hot', 'No'],
    ['Sunny', 'Cool', 'Yes'],
    ['Rainy', 'Hot', 'Yes'],
    ['Rainy', 'Cool', 'Yes'],
    ['Cloudy', 'Hot', 'Yes'],
    ['Cloudy', 'Cool', 'Yes']
]

# Calculate Entropy
def entropy(labels):
    total = len(labels)
    counts = Counter(labels)

    result = 0

    for count in counts.values():
        p = count / total
        result -= p * math.log2(p)

    return result

# Calculate Information Gain
def information_gain(data, feature):
    labels = [row[-1] for row in data]
    total_entropy = entropy(labels)

    values = set(row[feature] for row in data)
    weighted_entropy = 0

    for value in values:
```

```
        subset = [row for row in data if row[feature] == value]
        subset_labels = [row[-1] for row in subset]

        weighted_entropy += (
            len(subset) / len(data)
        ) * entropy(subset_labels)

    return total_entropy - weighted_entropy

# Build Decision Tree
def build_tree(data, features):
    labels = [row[-1] for row in data]

    # If all labels are same
    if len(set(labels)) == 1:
        return labels[0]

    # If no features remain
    if not features:
        return Counter(labels).most_common(1)[0][0]

    # Select best feature
    best_feature = max(
        features,
        key=lambda f: information_gain(data, f)
    )

    tree = {best_feature: {}}

    values = set(row[best_feature] for row in data)

    for value in values:
        subset = [row for row in data if row[best_feature] == value]

        if subset:
            remaining = [
                f for f in features if f != best_feature
            ]

            tree[best_feature][value] = build_tree(
                subset, remaining
            )

    return tree
```

```
# Main program
features = [0, 1]

tree = build_tree(data, features)

print("Decision Tree:")
print(tree)
```

Output: